Enlace a github: https://github.com/iperez288/market26

Aday Pérez

- "Write short units of code"
  - **Código Inicial:**

```
    public boolean doPurchase(String email, ProposedSale ps) {
        Buyer comprador;
        Seller vendedor;
        ProposedSale venta;
        Sale s;
        float precio;

        Transaction cobroComprador;
        Transaction pagoVendedor;


        Date today = UtilDate.trim(new Date());

        venta=ps;
        precio=venta.getPrice();

        db.getTransaction().begin();

        //Voy a modificar ProposedSale, Sale, Vendedor y Comprador.

        //1 Quitarle el dinero al comprador
        String bmail = ps.getBuyer().getEmail();
        comprador = (Buyer) getUser(bmail);

        //Creo la transacci n
        cobroComprador = new Transaction(precio,today,TransactionType.purchase,comprador);

        //al comprador, llamo un metodo para que a ade la transacci n y modifique el saldo seg n toque
        comprador.addTransaction(cobroComprador);

        db.persist(cobroComprador);

    // 2 Marcar la compra como vendida

        venta = getProposedSale(ps.getID());
        venta.setFechaCompra(today);

        int sID = venta.getSale().getSaleNumber();
        s= db.find(Sale.class, sID);
        s.doPurchase();



    // 3 Darle el dinero al vendedor
        vendedor = (Seller) getUser(email);
        pagoVendedor = new Transaction(precio,today,TransactionType.sale,vendedor);
        vendedor.addTransaction(pagoVendedor);
        db.persist(pagoVendedor);

        db.getTransaction().commit();

        return true;
    }
```

  - **Codigo refactorizado**

```
public boolean doPurchase(String email, ProposedSale ps) {
    db.getTransaction().begin();

    cobrarAlComprador(ps);
    marcarCompraVendida(ps);
    darDineroVendedor(email,ps);
    db.getTransaction().commit();

    return true;
}

private void cobrarAlComprador(ProposedSale ps) {
    Date today = UtilDate.trim(new Date());

    float precio=ps.getPrice();
    String bmail=ps.getBuyer().getEmail();
    Buyer comprador=(Buyer)getUser(bmail);
    Transaction cobroComprador=new Transaction(precio,today,TransactionType.purchase,comprador);
    comprador.addTransaction(cobroComprador);
    db.persist(cobroComprador);
}

private void marcarCompraVendida(ProposedSale ps) {
    Date today = UtilDate.trim(new Date());

    ProposedSale venta=getProposedSale(ps.getID());
    venta.setFechaCompra(today);
    int sID=venta.getSale().getSaleNumber();
    Sale s1=db.find(Sale.class,sID);
    s1.doPurchase();
    }

private void darDineroVendedor(String email,ProposedSale ps) {
    Date today = UtilDate.trim(new Date());
    Seller vendedor=(Seller) getUser(email);
    Transaction pagoVendedor=new Transaction(ps.getPrice(),today,TransactionType.sale,vendedor);
    vendedor.addTransaction(pagoVendedor);
    db.persist(pagoVendedor);
}
```

- **Descripción**:
  En el método doPurchase se hacían demasiadas tareas desde un mismo método, descontar dinero al comprador, marcar como vendido el producto y sumar el dinero al vendedor, sobrecargando el método y superando las 25 líneas. Por tanto se ha separado las tareas en distintos métodos auxiliares más cortos.
- **Miembro**: Aday Pérez

- "Write simple units of code"
  - **Código inicial:**

```
    }
    private String check_fields_Errors() {

        try {
            if ((fieldTitle.getText().length()==0) || (fieldDescription.getText().length()==0)  || (jTextFieldPrice.getText().length()==0))
                return ResourceBundle.getBundle("Etiquetas").getString("CreateSaleGUI.ErrorQuery");
            else {

                // trigger an exception if the introduced string is not a number
                    float price = Float.parseFloat(jTextFieldPrice.getText());
                    if (price <= 0)
                        return ResourceBundle.getBundle("Etiquetas").getString("CreateSaleGUI.PriceMustBeGreaterThan0");

                    else
                        return null;
            }
        } catch (java.lang.NumberFormatException e1) {

            return  ResourceBundle.getBundle("Etiquetas").getString("CreateSaleGUI.ErrorNumber");
        } catch (Exception e1) {
            e1.printStackTrace();
            return null;

        }
    }
```

  - **Código refactorizado**

```
    private String check_fields_Errors() {
            if (comprobarTextos())
                return ResourceBundle.getBundle("Etiquetas").getString("CreateSaleGUI.ErrorQuery");

            return comprobarNumero();
    }

    private boolean comprobarTextos() {
        return (fieldTitle.getText().length()==0) || (fieldDescription.getText().length()==0)  || (jTextFieldPrice.getText().length()==0 );
    }

    private String comprobarNumero() {
        try {
            if(Float.parseFloat(jTextFieldPrice.getText())<=0) {
                return ResourceBundle.getBundle("Etiquetas").getString("CreateSaleGUI.PriceMustBeGreaterThan0");
            }else {
                return null;
            }

        }catch(java.lang.NumberFormatException e1) {
            return  ResourceBundle.getBundle("Etiquetas").getString("CreateSaleGUI.ErrorNumber");
        }catch (Exception e1) {
            e1.printStackTrace();
            return null;
        }
    }
```

  - **Descripción**
    El método check_fields_Errors(), de la clase CreateSaleGUI, comprueba que los campos de textos y el precio sean correctos. Para ello tiene una complejidad ciclomatica de 7 (3 en el if para los textos, 1 para el precio, 2 catch) por tanto, queda demasiado complejo. Para ello se extraen las comprobaciones, analizando por separado los textos y el precio en métodos auxiliares. Dejando el método original únicamente con dos caminos posibles.
  - **Miembro**: Aday Pérez

- “Duplicte code”
  - **Código inicial:**

```
386
387     public void annadirSaldo(String email, float importe) {
388
389
390         db.getTransaction().begin();
391
392         Buyer b = db.find(Buyer.class, email);
393
394         Transaction t = new Transaction(importe, new Date(), Transaction.TransactionType.income, b);
395
396         b.addTransaction(t);
397
398         db.persist(t);
399
400         db.getTransaction().commit();
401
402     }
404     public void retirarSaldo(String email, float importe) {
405
406         User u = db.find(User.class, email);
407
408         db.getTransaction().begin();
409
410         Buyer b = (Buyer) u;
411
412         Transaction t = new Transaction(importe, new Date(), Transaction.TransactionType.extraction, b);
413
414         b.addTransaction(t);
415
416         db.persist(t);
417
418         db.getTransaction().commit();
419
420     }
```

  - **Código refactorizado:**

```
386
387     public void annadirSaldo(String email, float importe) {
388         modificarSaldo(email,importe,Transaction.TransactionType.income);
389     }
390
391     public void retirarSaldo(String email, float importe) {
392         modificarSaldo(email, importe, Transaction.TransactionType.extraction);
393     }
394
395     private void modificarSaldo(String email, float importe, Transaction.TransactionType transaction) {
396         db.getTransaction().begin();
397
398         Buyer b = db.find(Buyer.class, email);
399         Transaction t = new Transaction(importe, new Date(), transaction, b);
400         b.addTransaction(t);
401
402         db.persist(t);
403         db.getTransaction().commit();
404     }
```

  - **Descripción:** Los métodos annadirSaldo y retirarSaldo de la clase DataAccess realizan la misma acción menos por el tipo de transacción(uno income y otro extraction) pero el resto de las líneas se están duplicando. Por tanto, se han extraído a un método genérico, modificarSaldo, que recibe los mismos parámetros además del tipo de Transaccion, y que actúa según esto.

  - **Miembro**: Aday Pérez

- “keep unit interfaces small”
    - **Código inicial:**

```
    public Sale(String title, String description, int status, float price, Date pubDate, File file, Seller seller) {
        super();

        this.proposedSales=new ArrayList<ProposedSale>();
        this.title = title;
        this.description = description;
        this.status = status;
        this.price=price;
        this.pubDate=pubDate;
        if (file!=null) {
            this.fileName=file.getName();
            try {
                BufferedImage img1 = ImageIO.read(file);

                String path="src/main/resources/images/";
                File outputfile = new File(path+file.getName());


               ImageIO.write(img1, "png", outputfile);  // ignore returned boolean

            } catch(IOException ex) {
                //System.out.println("Write error for " + outputfile.getPath()  ": " + ex.getMessage());
        }
        }

        this.seller = seller;
        this.purchased=false;
        this.conversaciones = new ArrayList<Conversacion>();
    }
```

    - **Código refactorizado:**

```
    public Sale(SaleParameter parameterObject, Date pubDate, File file, Seller seller) {
        super();

        this.proposedSales=new ArrayList<ProposedSale>();
        this.title = parameterObject.getTitle();
        this.description = parameterObject.getDescription();
        this.status = parameterObject.getStatus();
        this.price=parameterObject.getPrice();
        this.pubDate=pubDate;
        if (file!=null) {
            this.fileName=file.getName();
            try {
                BufferedImage img1 = ImageIO.read(file);

                String path="src/main/resources/images/";
                File outputfile = new File(path+file.getName());


               ImageIO.write(img1, "png", outputfile);  // ignore returned boolean

            } catch(IOException ex) {
                //System.out.println("Write error for " + outputfile.getPath()  ": " + ex.getMessage());
        }
        }

        this.seller = seller;
        this.purchased=false;
        this.conversaciones = new ArrayList<Conversacion>();
    }
```

```
public class SaleParameter {
    private String title;
    private String description;
    private int status;
    private float price;

    public SaleParameter(String title, String description, int status, float price) {
        this.title = title;
        this.description = description;
        this.status = status;
        this.price = price;
    }
```

    - **Descripción:**

      El contructor parametrizado de la clase Sale necesita 7 parametros, lo que aumenta la complejidad, por tanto se ha creado una nueva clase llamada SaleParameter que tiene cuatro de esos parametros con sus getter y setter, de esta manera solo necesita 4 parametros, haciendolo más comprensible y simple. Tambien afecta a los que

usan el constructor en que a la hora de pasar los parametros algunos de ellos los deben poner en un nuevo objeto SaleParameter.

- **Miembro**: Aday Pérez

Iker Perez

“Write short units of code”

Código inicial (método `hacerValoracion`, DataAccess):

```
406    public void hacerValoracion(int pID, String email, int rate, String text) {
407
408            Valoracion val;
409            Buyer u;
410            ProposedSale ps;
411            long numV; //Número de valoraciones que tiene el vendedor.
412            float sum; //Suma de valoraciones del vendedor
413
414            //Traer de vuelta la Sale y el email
415
416            db.getTransaction().begin();
417
418            //Se da por sentado que ya se ha comprobado antes que es comprador.
419            u = (Buyer) getUser(email);
420            ps = getProposedSale(pID);
421            val = new Valoracion(ps,u,rate,text);
422            ps.setValoracion(val);
423            u.addValoracion(val);
424            db.persist(val);
425
426            Seller s;
427
428            s = db.find(Seller.class, ps.getSeller());
429
430            /*TypedQuery<Seller> q1= db.createQuery("SELECT sl FROM Sale s JOIN s.seller sl WHERE  s.saleNumber = ?1", Seller.class);
431            q1.setParameter(1, snum);
432
433            List<Seller> sl = q1.getResultList();
434            if(sl.size()>1) {
435                System.out.println("Más de un vendedor encontrado");
436            }
437            s = sl.get(0);*/
438
439            TypedQuery<Long> q2= db.createQuery("SELECT count(ps) FROM Seller sl JOIN sl.sales s JOIN s.proposedSales ps WHERE s.purchased = TRUE /
440            q2.setParameter(1, s.getEmail());
441
442            numV = q2.getResultList().get(0);
443
444            System.out.println("Num valoraciones de seller "+ s.getEmail() + ": " + numV);
445
446            sum = s.getRate()*numV;
447            sum = sum + rate;
```

Código refactorizado:

```
406    public void hacerValoracion(int pID, String email, int rate, String text) {
407        db.getTransaction().begin();
408
409        //Se da por sentado que ya se ha comprobado antes que es comprador.
410        Buyer u = (Buyer) getUser(email);
411        ProposedSale ps = getProposedSale(pID);
412        Valoracion val = new Valoracion(ps, u, rate, text);
413        ps.setValoracion(val);
414        u.addValoracion(val);
415        db.persist(val);
416
417        actualizarPuntuacionVendedor(ps, rate);
418
419        db.getTransaction().commit();
420    }
421
422    private void actualizarPuntuacionVendedor(ProposedSale ps, int rate) {
423        Seller s = db.find(Seller.class, ps.getSeller());
424        long numV = contarValoraciones(s.getEmail());
425
426        float sum = s.getRate() * numV + rate;
427        s.setRate(sum / (numV + 1));
428    }
429
430    private long contarValoraciones(String sellerEmail) {
431        TypedQuery<Long> q = db.createQuery("SELECT count(ps) FROM Seller sl JOIN sl.sales s JOIN s.proposedSales ps WHERE s.purchased = TRUE AND
432        q.setParameter(1, sellerEmail);
433
434        return q.getResultList().get(0);
435    }
```

Descripción:

El método hacerValoracion de la clase DataAccess hacía demasiadas tareas a la vez: crear y guardar la valoración, contar las valoraciones que ya tenía el vendedor y recalcular su puntuación media. Esto lo hacía muy largo y superaba las 25 líneas. Por tanto, se han separado las tareas en dos métodos auxiliares más cortos: `contarValoraciones`, que hace la consulta, y `actualizarPuntuacionVendedor`, que calcula la nueva media.

Miembro: Iker Pérez

"Write simple units of code"

Código inicial (constructor `DataAccess()`):

```
59    public DataAccess() {
60        if (c.isDatabaseInitialized()) {
61            String fileName = c.getDbFilename();
62
63            if (!c.isDatabaseLocal())
64                fileName = DBSERVERDIB + fileName;
65
66            try {
67                Files.delete(Paths.get(fileName));
68
69                Files.deleteIfExists(Paths.get(fileName + "$"));
70
71                System.out.println("File deleted");
72            } catch (IOException e) {
73                System.out.println("Operation failed: " + e.getMessage());
74            }
75        }
76        open();
77        if (c.isDatabaseInitialized())
78            initializeDB();
79        System.out.println("DataAccess created => isDatabaseLocal: " + c.isDatabaseLocal() + " isDatabaseInitialized: "
80                + c.isDatabaseInitialized());
81
82        close();
83
84    }
```

Código refactorizado:

```
59    public DataAccess() {
60        if (c.isDatabaseInitialized())
61            borrarFicheroBD();
62
63        open();
64        if (c.isDatabaseInitialized())
65            initializeDB();
66        System.out.println("DataAccess created => isDatabaseLocal: " + c.isDatabaseLocal() + " isDatabaseInitialized: "
67                + c.isDatabaseInitialized());
68
69        close();
70    }
71
72    private void borrarFicheroBD() {
73        String fileName = rutaFicheroBD();
74        try {
75            Files.delete(Paths.get(fileName));
76            Files.deleteIfExists(Paths.get(fileName + "$"));
77            System.out.println("File deleted");
78        } catch (IOException e) {
79            System.out.println("Operation failed: " + e.getMessage());
80        }
81    }
82
83    private String rutaFicheroBD() {
84        if (c.isDatabaseLocal())
85            return c.getDbFilename();
86        return DBSERVERDIB + c.getDbFilename();
87    }
88
89    public DataAccess(EntityManager db) {
90        this.db = db;
91    }
```

Descripción:
El constructor de la clase DataAccess tiene una complejidad ciclomática de 6 (1 del camino base, 3 de los if, 1 del if de la ruta y 1 del catch), por lo que queda demasiado complejo. Para solucionarlo se extrae el borrado del fichero a un método auxiliar `borrarFicheroBD`, y el cálculo de la ruta a otro método, `rutaFicheroBD`. De esta forma el constructor queda con una complejidad de 3 y cada método auxiliar con una de 2.

Miembro: Iker Pérez

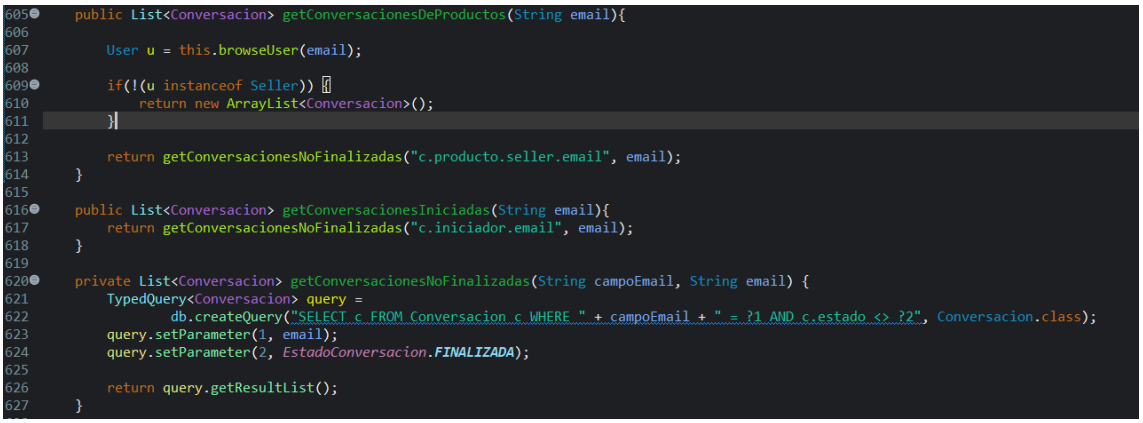
"Duplicate code"

Código inicial:



```
605    public List<Conversacion> getConversacionesDeProductos(String email){
606
607        User u = this.browseUser(email);
608
609        if(!(u instanceof Seller)) {
610            return new ArrayList<Conversacion>();
611        }
612
613        TypedQuery<Conversacion> query =
614                db.createQuery("SELECT c FROM Conversacion c WHERE c.producto.seller.email = ?1 AND c.estado <> ?2", Conversacion.class);
615        query.setParameter(1, email);
616        query.setParameter(2, EstadoConversacion.FINALIZADA);
617
618        return query.getResultList();
619
620    }
621
622    /**
623     *
624     * @param email
625     * @return Conversaciones iniciadas por el usuario con correo electr⊕nico email.
626     */
627    public List<Conversacion> getConversacionesIniciadas(String email){
628
629        TypedQuery<Conversacion> query =
630                db.createQuery("SELECT c FROM Conversacion c WHERE c.iniciador.email = ?1 AND c.estado<>?2", Conversacion.class);
631        query.setParameter(1, email);
632        query.setParameter(2, EstadoConversacion.FINALIZADA);
633
634        return query.getResultList();
635    }
```

Código refactorizado:



```
605    public List<Conversacion> getConversacionesDeProductos(String email){
606
607        User u = this.browseUser(email);
608
609        if(!(u instanceof Seller)) {
610            return new ArrayList<Conversacion>();
611        }
612
613        return getConversacionesNoFinalizadas("c.producto.seller.email", email);
614    }
615
616    public List<Conversacion> getConversacionesIniciadas(String email){
617        return getConversacionesNoFinalizadas("c.iniciador.email", email);
618    }
619
620    private List<Conversacion> getConversacionesNoFinalizadas(String campoEmail, String email) {
621        TypedQuery<Conversacion> query =
622                db.createQuery("SELECT c FROM Conversacion c WHERE " + campoEmail + " = ?1 AND c.estado <> ?2", Conversacion.class);
623        query.setParameter(1, email);
624        query.setParameter(2, EstadoConversacion.FINALIZADA);
625
626        return query.getResultList();
627    }
```

Descripción:

Los métodos getConversacionesDeProductos y getConversacionesIniciadas de la clase DataAccess realizan la misma consulta, salvo por el campo por el que se filtra

el email (el vendedor del producto o el iniciador de la conversación). El resto de las líneas (crear la consulta, asignar los parámetros y devolver el resultado) estaban duplicadas. Por tanto, se han extraído a un método genérico, `getConversacionesNoFinalizadas`, que recibe el campo por el que filtrar además del email.

Miembro: Iker Pérez